Finance-RAG
Implementation Report

Github Link: https://github.com/Jaysenso/finance-rag

By: **J'sen Ong**

(Jsenongxuan@gmail.com)

Table of Contents

1. Introduction **3**

1.1 Infrastructure Limitations 3

1. Data Processing Pipeline **4**

2.1 Data Collection 4

2.2 Metadata Extraction 4

2.3 Docling Parsing Process 5

2.4 Chunking Strategy 6

2. Retrieval Architecture **7**

3.1 Query Processing pipeline 7

3.2 Retrieval Strategy: HyPE & Reciprocal Rank Fusion 9

3.3 Reranking 10

3. Evaluation **10**

4.1 Evaluation Methodologies 10

4.2 RAGAS Results 11

4.2.1 Result Anomaly 12

4.3 Ablation Study: RAG Against LLM-Only and Web Search Baselines 15

4.3.1 Ablation Study Methodologies 15

4.3.2 Model Selection and Experimental Conditions 15

4.3.3 Test Set Construction 16

4.3.4 Evaluation Modes 16

4.3.5 Acknowledged Limitations 17

4.4 Ablation Results 18

4.4.1 LLM-as-Judge Prompts 18

4.4.2 Scoring Metrics 18

4.4.3 Results Analysis 19

1. Introduction

This system is a Retrieval-Augmented Generation (RAG) pipeline built for querying public companies' SEC financial filings. It ingests annual, quarterly, and current event reports directly from EDGAR, processes them through a Docling Document parser, and indexes the content into a Qdrant cloud-hosted vector database. At query time, a LangGraph orchestration agent decomposes user questions, retrieves relevant chunks through a dual-database fusion strategy, reranks them with a cross-encoder, verifies their adequacy with an LLM, and self-corrects through reformulation if needed before generating a grounded and cited answer.

1.1 Infrastructure Limitations

In this implementation, all language models are accessed through free API endpoints provided via OpenRouter, while embeddings are generated using locally hosted embedding models running on limited computational resources. These constraints introduce practical limitations in both compute capacity and cost, which influenced several design decisions throughout the system.

As a result, model selection prioritizes throughput and inference speed rather than exhaustive model optimization. The chosen models are sufficiently capable and readily available through OpenRouter, allowing the pipeline to remain functional within free tier usage limits. For this reason, the following sections do not focus extensively on model comparison or justification, and instead emphasize the design and implementation of the overall pipeline.

The objective of this project is to demonstrate that a well designed retrieval and processing pipeline can achieve a reasonable level of accuracy and usable results even under constrained infrastructure conditions. In a production environment, whether through corporate API subscriptions or deployment of higher capacity on premise models, performance and output quality can reasonably be expected to improve beyond the results obtained in this implementation.

1. Data Processing Pipeline

2.1 Data Collection

The system ingests SEC financial filings, specifically 10 K (annual), 10 Q (quarterly), and 8 K (current event) reports, for a set of public companies. In the current implementation, data collection was completed for nine public companies, each with a varying number of available filings across 10 K, 10 Q, and 8 K reports. This limitation exists because the implementation relies on SecAPI, a paid service, and data extraction was performed using a free API key with restricted usage limits. As a result, only nine companies were successfully scrapped within the allowed quota. These filings can be obtained through two data sourcing pathways.

The first pathway is automated SEC scraping via sec_scrapper.py, which uses the sec-api Python package to search EDGAR for filings from 2023 to 2026 and convert them into PDF format. Each company retrieves up to five filings per document type, which are stored in src/data/pdf/{TICKER}/{DOC_TYPE}/. The second pathway allows manual placement of files into src/data/filings_to_process/ for batch ingestion. All files included in the current dataset were scraped directly from the SEC; however, the system also supports ingestion of external documents by placing them into this folder for processing. For more details on the ingestion process, refer to the Readme.md in the Github repository.

2.2 Metadata Extraction

JPM_10-K_0000019617-25-000270	11/02/2026 9:29 PM	Adobe Acrobat D...	16,479 KB
META_10-K_0001326801-25-000017	11/02/2026 9:19 PM	Adobe Acrobat D...	4,600 KB
MSFT_8-K_0001193125-25-225125	11/02/2026 9:09 PM	Adobe Acrobat D...	98 KB
MSFT_8-K_0001193125-25-256310	11/02/2026 9:09 PM	Adobe Acrobat D...	127 KB

Metadata is extracted from filenames using extract_metadata_from_filename in utils.py. The expected naming convention follows the format COMPANY_DOCTYPE_ACCESSION.pdf, where the two digit year contained in the accession number is expanded into a four digit year. Filing dates are currently approximated, with 10 K files assigned a date of December 31 and 10 Q files assigned a date of September 30. Each generated chunk stores metadata including company, document type, filing date, page number, chunk index, parent document ID and so on.

The current implementation represents an extreme simplification of the metadata input process. Metadata is inferred purely from file naming conventions rather than sourced directly from authoritative filing information. In a production level system with a web based interface, metadata would more realistically be supplied through structured mechanisms such as a user interface that allows users to input or verify company details, automated extraction from SEC API responses, or other ingestion workflows designed to ensure higher accuracy and completeness.

These metadata fields play an important role in downstream retrieval and filtering, and their usage and significance will be further elaborated in the retrieval section.

2.3 Docling Parsing Process

The parser supports three file formats. PDF, DOCX, and HTML files are processed using the Docling library, which performs GPU accelerated table structure recognition and image extraction. Excel files are parsed with pandas and openpyxl, where each worksheet is converted into a markdown table element. Images encountered during parsing are described using a vision LLM model.

When handling PDF and DOCX files, Docling extracts document content through a hierarchical element tree. When a table element is detected, it is automatically extracted and converted into markdown format. In addition to this default behavior, the implementation generates an image representation of each detected table and introduces an **optional** feature that allows users to invoke a vision LLM inference for supplementary analysis. When enabled, the vision model receives both the table image and the markdown generated by Docling, and can provide additional contextual descriptions as well as correct or refine the markdown representation. In the current production configuration, **"use_vision_for_tables"** is set to false, meaning table OCR correction is disabled and only image description is active.

This feature introduces **significant computational overhead** because enabling it requires every detected table to be processed by the vision LLM. Given the large number of tables typically present in SEC filings, this substantially increases processing time and resource consumption. Nevertheless, when extraction accuracy is a higher priority than computational efficiency, enabling this feature represents a deliberate and justifiable tradeoff.

2.4 Chunking Strategy

The SemanticChunker in chunking.py implements a structure aware chunking strategy designed for financial documents. Chunk size is set to 1,024 tokens to balance contextual completeness with embedding quality, while a 256 token overlap preserves continuity between adjacent chunks. A minimum size of 512 tokens avoids producing small, low value fragments, and a maximum limit of 2,048 tokens ensures compatibility with embedding model constraints.

Despite its name, SemanticChunker does not rely on an LLM or embedding based semantic splitting. Instead, it follows a rule based approach. The chunker sequentially slides through document elements and appends text into a buffer until the target token size is reached. Once the limit is met, the buffer is emitted as a chunk, and the process continues with an overlap window to maintain context across boundaries.

This design works effectively because Docling already extracts documents using a hierarchical element structure. Sections, paragraphs, tables, and images are identified as structured elements, meaning much of the semantic coherence is preserved during parsing.

Tables and images are handled slightly differently. Each is assigned its own chunk and enriched with nearby text elements that act as contextual captions, since financial tables and images are often difficult to interpret without surrounding explanations. Regular text content is merged paragraph by paragraph as the buffer grows, and small trailing sections are merged into the previous chunk rather than emitted as standalone fragments.

Overall, this rule based sliding window approach leverages the strong structural information produced by Docling, achieving semantically coherent chunks while avoiding the additional computational overhead of LLM driven chunking.

2. Retrieval Architecture

3.1 Query Processing pipeline

The query pipeline is orchestrated as a LangGraph state graph in **RAGAgent** class, following this structure:

```
START → analyze → retrieve → rerank → verify → decide_next
                                        ├→ reformulate → retrieve (retry loop)
                                        ├→ retrieve (next sub-query)
                                        └→ generate → END
```

```
================================================================================
NODE: ANALYZE
================================================================================
User Query: How are companies in the technology sector discussing AI adoption in their latest filings?
Intent: general
Companies: ['NVDA', 'MSFT', 'META', 'TSLA', 'AVGO', 'AMZN', 'AAPL']
Document Types: ['10-K', '10-Q', '8-K']
Sub-queries: 7
  1. AI adoption discussion (company=NVDA, doc_type=None)
  2. AI adoption discussion (company=MSFT, doc_type=None)
  3. AI adoption discussion (company=META, doc_type=None)
  4. AI adoption discussion (company=TSLA, doc_type=None)
  5. AI adoption discussion (company=AVGO, doc_type=None)
  6. AI adoption discussion (company=AMZN, doc_type=None)
  7. AI adoption discussion (company=AAPL, doc_type=None)
```

The **Analyze node** uses an LLM to decompose the user query into structured sub query objects and infer which company the user is referring to as well as the relevant document type implied by the question. While filters can theoretically be made more fine grained to precisely target specific filings or time ranges, the current implementation intentionally keeps filtering relaxed. This design choice avoids the risk of excluding relevant documents due to incorrect assumptions made during LLM based query interpretation. Only company and document type are used as filtering dimensions, allowing the system to preserve recall and avoid prematurely filtering out potentially relevant information.

Before query analysis begins, the system queries all existing points stored in Qdrant to determine which companies have already been ingested into the vector database. This list of available companies is then injected into the analysis agent prompt, allowing the LLM to select from known entities rather than hallucinating unsupported companies.

Because downstream stages include reranking and Reciprocal Rank Fusion (RRF) ranking, the system can afford to intentionally cast a wide retrieval net. The Retrieve node applies HyPE hybrid retrieval when enabled, falling back to standard content-only search otherwise. It retrieves up to 30 candidate documents per sub query with a minimum score threshold of 0.5. The broader candidate set prioritizes recall at this stage, while the reranker and RRF components perform the heavier relevance filtering later in the pipeline, determining which contexts are ultimately most useful for answering the user query. HyPE and RRF are elaborated in the following sections.

```
================================================================================
NODE: VERIFY
================================================================================
Verification Score: 0.90
Verified: True
Reason: The context provides detailed information about JPMorgan's common stock share repurchase
program, including the $30 billion authorization in 2024, quarterly repurchase amounts, and total
shares repurchased for 2024, 2023, and 2022.
Missing: The context does not include the specific share repurchase data for individual months within
2024 beyond the quarterly breakdown.
```

The **Verify node** uses an LLM to judge whether the retrieved context adequately answers the sub-query, returning a score between 0.0 and 1.0. The passing threshold is currently set to 0.70 in **config.yaml**, which has proven optimal through empirical testing, though in production this parameter is expected to vary depending on the nature of the data users are ingesting.

Threshold calibration has meaningful downstream consequences. A threshold set too high risks triggering the retry operation repeatedly when the retriever cannot surface context of sufficient quality, accumulating overhead with each attempt. This becomes particularly costly in worst-case scenarios where a user queries a sector or set of companies that are simply not well represented in the knowledge base. In those situations the query is unanswerable by the available context, yet the system will exhaust its maximum retry count before surfacing that conclusion. With the number of sub-queries the pipeline generates, this failure mode compounds quickly across the graph and can produce a severe degradation in response time that directly impacts user experience. The threshold therefore needs to be tuned with the expected knowledge base coverage in mind, balancing retrieval quality against the overhead of aggressive retrying.

```
================================================================================
NODE: REFORMULATE
================================================================================
Original Query: How did U.S. export restrictions affect NVIDIA's business performance in Q3 2023?
Reformulated Query: What specific impact did U.S. export restrictions have on NVIDIA's Q3 2023
business performance and revenue?
```

The **Reformulate node** activates when verification fails and retries remain (maximum of two). It uses the verification feedback, including the failure reason and missing information fields, to rewrite the query, tracking reformulation history to avoid loops.

The **Decide Next node** routes the pipeline to reformulate and retry, process the next sub-query, or proceed to final answer generation. The two-retry ceiling limits latency while giving the system a meaningful opportunity to recover from poor initial retrievals.

3.2 Retrieval Strategy: HyPE & Reciprocal Rank Fusion

The most architecturally distinctive feature of the system is HyPE, a dual-database retrieval strategy implemented across **hybrid_retriever**.py and **hype_ingestion**.py.

HyPE is a deliberate evolution of HyDE (Hypothetical Document Embeddings). HyDE generates hypothetical documents at query time, meaning every user query incurs an LLM call before retrieval can begin, adding latency directly to the critical path. Given that the pipeline already performs embedding, multi-collection search, iterative query reformulation and retry, and RRF fusion at query time, absorbing an additional LLM generation step would compound latency in a way that degrades the interactive experience. HyPE shifts that generation overhead entirely to ingestion, where it is a one-time, offline cost per chunk. The tradeoff is worthwhile because ingestion is a batch process and happens far from the user. Query time, by contrast, is where latency is felt directly.

At query time, query analysis first extracts keywords from the user query rather than using the raw query directly, as keywords tend to yield stronger semantic similarity matches in a vector database. These keywords are then embedded once and used to search both the content collection and the question collection, fetching up to twice the retrieval limit in the worst case. If the initial retrieval yields no results, query reformulation kicks in to rephrase and retry the search. Question

collection results are mapped back to their parent chunks, and the two result sets are fused using Reciprocal Rank Fusion with k=60 and equal weights of 0.5 for each source. RRF was chosen over score averaging because scores from the content and question collections are not directly comparable and come from different embedding spaces, making a rank-based, score-agnostic fusion strategy the appropriate choice.

The underlying rationale for HyPE is that user queries like "What was Apple's revenue?" are semantically closer to a hypothetical question like "What was Apple's total net revenue for fiscal year 2024?" than to the raw financial prose those answers appear in. This is particularly valuable in the financial domain, where document language is formal and terminology-dense.

3.3 Reranking

After initial retrieval, the pipeline aggregates a large number of candidate chunks across multiple sub-queries, making a second filtering stage essential. Without it, the final generation LLM would be flooded with noisy, redundant, or weakly relevant context, which at sufficient volume can actively degrade response quality and introduce hallucinations. Reranking addresses this by scoring each query and chunk pair directly using cross-attention through a cross-encoder, keeping the top 15 results. Because cross-encoders jointly encode the query and chunk together rather than embedding them independently, they produce significantly more accurate relevance judgements than the other filters operating upstream, making this the highest-fidelity filtering step in the pipeline.

3. Evaluation

4.1 Evaluation Methodologies

The primary evaluation uses the RAGAS library with metrics for context precision, context recall, and faithfulness. An async scoring pipeline runs each sample through the RAG agent, collects retrieved contexts and generated answers, and scores them against ground truth. The test set in **test_set.json** is synthetically generated by **generate_dataset.py,** which randomly samples chunks from Qdrant and uses an LLM to produce question, ground truth, and context triplets. This is a

practical approach given the absence of an annotated dataset, though it introduces a potential circularity where questions generated from the same corpus the system indexes may be biased toward content the retriever already handles well. A larger and more diverse test set covering numerical reasoning, cross-document comparison, and hallucination resistance would strengthen confidence in the results.

Any evaluation that excludes human judgement has inherent limitations in how meaningful its conclusions can be. Synthetic and automated metrics are best treated as a baseline reference rather than a definitive measure of system quality, which is a reasonable standard for RAG evaluation in the absence of annotated data.

To supplement these limitations, the system includes an ablation study comparing three configurations: the full RAG pipeline, the same LLM operating without any retrieval context, and an external model with web search access. This allows the contribution of retrieval to be isolated and compared against both a no-context baseline and an alternative information source, giving the results more interpretive weight than a single configuration evaluation would provide.

4.2 RAGAS Results

The synthetic data generation randomly sampled 50 chunks and generated question-answer pairs for evaluation using the RAGAS framework across the three metrics above. Due to throughput latency and the time required to process each sample through the full pipeline, only **20** chunks were evaluated to completion.

Metrics	Scores
Context Precision	0.6284615224
Context Recall	0.8998611111
Faithfulness	0.8891812865

The results reveal a retrieval system with mixed performance characteristics. Context recall at **0.90** indicates the system successfully retrieves relevant information when it exists, meaning various search strategies are generally effective at surfacing the correct chunks. Faithfulness at **0.89** shows the generation model reliably grounds its answers in the provided context rather than hallucinating or introducing unsupported claims, which is critical for a financial domain where factual accuracy is non-negotiable.

However, context precision at **0.63** exposes what appears to be a significant weakness, though it is largely an expected artifact of the system's design constraints. This metric measures what fraction of the retrieved context is actually relevant to answering the query. A score of **0.63** means that roughly **37%** of the chunks passed to the generation model are noise, either tangentially related or outright irrelevant. This lower precision is inevitable given that the system cannot dynamically adjust thresholds for verification or retrieval limits. A trivial query asking for a single company's revenue may require only two or three highly relevant chunks, while a non-trivial query comparing growth rates across ten companies in a sector may legitimately require dozens of chunks to construct a complete answer. The pipeline is tuned towards over-retrieval to ensure it does not prematurely cut off necessary context for complex queries, which necessarily inflates the denominator for simpler queries where much of the retrieved material ends up unused. The verification threshold of 0.70 compounds this by allowing marginally relevant contexts to pass through when the system is uncertain, prioritizing coverage over surgical precision. The cost is twofold: the generation model must spend significant token budget processing irrelevant material, and in cases where irrelevant context is highly ranked, it risks diluting or derailing the final answer. The gap between recall and precision suggests the system prioritizes not missing critical information over maintaining a clean context set, which is a defensible trade-off for a financial QA system where an incomplete answer is often worse than a verbose one.

4.2.1 Result Anomaly

One notable failure case emerged during the result inspection. A query failed to retrieve any contexts, which was unexpected given that each generated question is derived from a chunk guaranteed to exist in the database. Further investigation revealed the root cause: the supporting chunk was extracted from a 10-K filing, but when the query was processed by the RAG pipeline,

the Query Analysis agent inferred that the information should reside in an 8-K filing and applied a metadata filter accordingly as shown below:


```
{
  "question": "What new export restrictions did the U.S. government announce
  in Q3 2023 that impact NVIDIA's business?",
  "ground_truth": "The U.S. government announced new export restrictions and
  licensing requirements targeting China's semiconductor and supercomputing
  industries, impacting exports of certain chips, software, hardware,
  equipment, and technology used to develop, produce, and manufacture
  certain chips to China and Russia.",
  "context": "During the third quarter of fiscal year 2023, the U.S.
  government, or USG, announced new export restrictions and export licensing
  requirements targeting China's semiconductor and supercomputing
  industries. These restrictions impact exports of certain chips, as well as
  software, hardware, equipment, and technology used to develop, produce,
  and manufacture certain chips, to China (including Hong Kong and Macau)
  and Russia...",
  "metadata": {
    "company": "NVDA",
    "document_type": "10-K",
    "filing_date": "2023-12-31",
    "page_number": 29,
    "chunk_index": 59,
    "parent_doc_id": "37519c97-51f1-4b09-8e1e-e0fdbb447182",
    "has_table": false,
    "has_chart": false,
    "token_count": 934
  },
```


```
================================================================================
=
RAG PIPELINE EXECUTION
================================================================================
=
Query: What new export restrictions did the U.S. government announce in Q3
2023 that impact NVIDIA's business?

================================================================================
=
NODE: ANALYZE
================================================================================
=
User Query: What new export restrictions did the U.S. government announce in
Q3 2023 that impact NVIDIA's business?
Intent: general
Companies: ['NVDA']
Document Types: ['8-K']
Sub-queries: 1
  1. U.S. export restrictions Q3 2023 NVIDIA business impact (company=NVDA,
  doc_type=8-K)

================================================================================
=
```

User query:
"What new export restrictions did the U.S. government announce in Q3 2023 that impact NVIDIA's business?"

The Query Analysis Agent interpreted the reference to a specific quarter as an indicator that the information would be found in a quarterly 8-K filing rather than an annual 10-K report. This filtered out the only chunk that supported the fact, resulting in zero retrieved contexts. RAGAS returned an error for this sample, though it was ultimately included in the final scoring with all three metrics scores set to 0.

This failure highlights a fundamental limitation of the Query Analysis Agent. Metadata filtering is a core feature of the system and provides meaningful value by reducing search space, improving precision, and allowing the pipeline to target specific document types when user intent is clear. However, the trade-off is that incorrect filter inference becomes a single point of failure. When the agent misclassifies the source document type, it does not retrieve weakly relevant results or rank the correct context lower, it eliminates the correct context entirely from consideration. This is a fatal failure mode because there is no fallback mechanism. The retriever never sees the excluded chunks, the reranker cannot recover them, and the generation step operates on an incomplete or empty context set. In domains like financial documents where similar information can appear across multiple filing types with different disclosure cadences, this brittleness is particularly consequential. A 10-K may contain quarterly information discussed retrospectively, an 8-K may reference annual trends, and the distinction is often contextual rather than structural. The agent's reliance on heuristics about what "should" be in each filing type introduces a failure surface that scales with the ambiguity of the query and the diversity of the document set. Addressing this will likely require either a confidence-based fallback where uncertain filter predictions allow broader retrieval, or a multi-stage approach where initial retrieval operates without metadata constraints and filtering is applied during reranking instead.

4.3 Ablation Study: RAG Against LLM-Only and Web Search Baselines

4.3.1 Ablation Study Methodologies

While RAGAS provides a structured, automated framework for measuring retrieval and generation quality across dimensions such as faithfulness, answer relevancy, and context precision, it evaluates the RAG pipeline in isolation. A high RAGAS score reflects internal consistency between retrieved context and generated output, but does not speak to whether the system produces meaningfully better answers than simpler, lower-cost alternatives that a user or organisation might realistically choose instead. The ablation study was designed to address this gap by evaluating the RAG system's practical utility relative to two readily available alternatives: the same LLM operating without any retrieval context, and an LLM with web search access. These configurations represent genuine substitutes a user might reach for in place of a purpose-built RAG pipeline over SEC filings. The relevant question is therefore not whether retrieval alone explains any performance difference, but whether the full RAG system as deployed produces answers that justify the investment in retrieval infrastructure, document ingestion, and prompt engineering over simpler approaches.

4.3.2 Model Selection and Experimental Conditions

For the web search condition, GPT-4o-mini with native search capabilities was used rather than the same model used in the RAG and LLM-only conditions. This decision reflects two practical considerations. OpenRouter's web search plugin for non-GPT, Anthropic, Perplexity, or xAI models is powered by Exa, a third-party search engine that does not reflect how end users realistically retrieve financial information. In practice, most users seeking SEC filing data would turn to Google, ChatGPT, or Gemini as their primary search interface. Additionally, while GPT-4o would be the preferred model across all conditions for consistency, its input cost of $0.15 per million tokens makes it prohibitive for the RAG and LLM-only conditions at the query volumes

required for this study. GPT-4o usage is therefore scoped to the web search condition only, where its native search integration is essential to simulating realistic user behaviour.

This introduces a model disparity between conditions. However, the web search condition is best interpreted not as a controlled ablation variable but as a practical real-world alternative baseline, representing what a user would experience querying a commercially available search-augmented LLM rather than a purpose-built financial RAG system.

An important aspect of the baseline configurations is that both the LLM-only and web search conditions were run without any system prompt, receiving only the raw question with no instructional framing. This was a deliberate choice to reflect realistic usage conditions, as a user querying an LLM or web search tool directly would not have access to a domain-tuned system prompt. This means the comparison is not a clean isolation of retrieval contribution, and any performance advantage observed in the RAG system may reflect the combined effect of retrieval and prompt engineering rather than retrieval alone. Within the scope of this study, the comparison is best interpreted as a practical end-to-end evaluation of whether the RAG pipeline, as a complete system, outperforms the realistic alternatives available to an end user.

4.3.3 Test Set Construction

The evaluation dataset was constructed by randomly sampling chunks from the vector database and prompting an LLM to generate a realistic financial query grounded in each chunk. This produced (question, reference) pairs where the reference is the source chunk the question was generated from.

4.3.4 Evaluation Modes

A core tension in this ablation is that the three systems under comparison have fundamentally asymmetric information access. RAG retrieves directly from the indexed corpus, the LLM-only baseline operates on pre-trained knowledge, and the web search baseline accesses live web data that may extend beyond the ingested filings. This asymmetry creates a structural bias when a single reference-grounded evaluation is used. Because the reference is drawn directly from the RAG system's own vector database, scoring against that reference inherently advantages RAG, as

the system being evaluated and the ground truth share the same source. A financially accurate answer produced by web search might present figures from a more recent filing, a different exhibit, or a differently formatted disclosure, and a judge comparing it against the reference may incorrectly penalize it for deviation rather than error.

To address this, two evaluation modes were implemented. The reference-anchored evaluation measures factual alignment with the specific retrieved chunk, and is most appropriate for assessing RAG's retrieval-augmented precision, though it is acknowledged to carry an inherent retrieval-source bias. The reference-free evaluation removes the ground truth anchor entirely and asks the judge to assess financial plausibility, internal consistency, and reasoning quality, providing a fairer basis for comparing web search and LLM-only outputs where neither system is penalized for not matching a specific chunk it had no access to. Together, these two modes form complementary views of system performance: one measuring grounded retrieval accuracy and one measuring answer quality in a source-agnostic setting.

4.3.5 Acknowledged Limitations

Several limitations of this evaluation design are worth stating explicitly before turning to the results. The reference-free mode trades verifiability for fairness, as the judge cannot confirm exact figures and must rely on plausibility heuristics. The model disparity between conditions means performance differences between RAG and the web search baseline cannot be attributed to retrieval alone. The web search condition is further constrained by a limit of five searches per query, which underrepresents the iterative, cross-referencing process a user performing real financial research would likely follow, meaning the web search baseline may not reflect the ceiling of what a web-augmented workflow could achieve. A more controlled future study would hold both the model and system prompt constant across all conditions to isolate retrieval contribution more cleanly. Within the scope of this study, the results are best interpreted as a directional comparison of practical system configurations rather than a precise empirical measurement of each approach's ceiling performance.

4.4 Ablation Results

4.4.1 LLM-as-Judge Prompts

Before interpreting the results, it is worth clarifying what each metric captures in this evaluation. The scoring rubric is defined in full within **prompts.py**, and it is encouraged to refer to that file for the exact criteria used to prompt the LLM judge for both the reference-anchored and reference-free evaluation modes.

4.4.2 Scoring Metrics

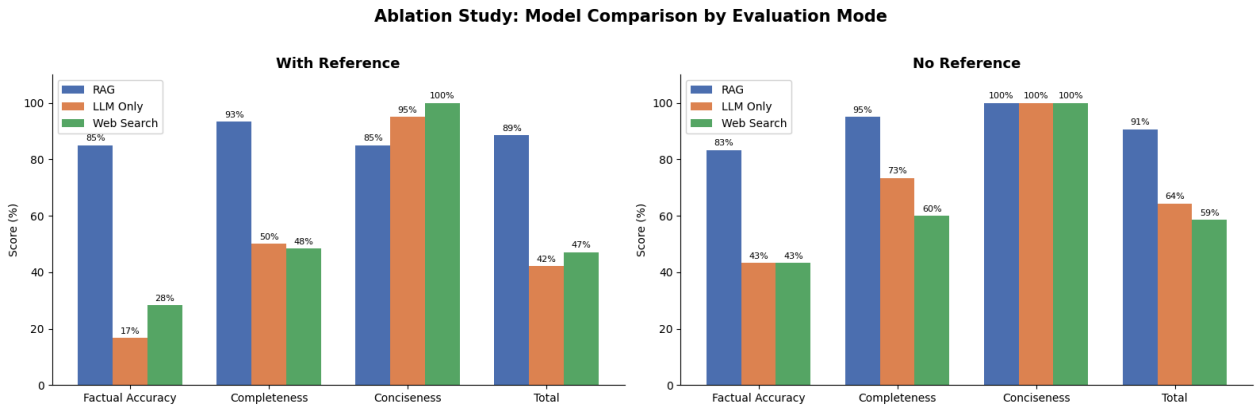
Factual Accuracy measures whether the financial figures, dates, and claims in the answer are correct and consistent with verified facts. In the reference-anchored mode, this is assessed directly against the retrieved chunk. In the reference-free mode, the judge has no ground truth anchor to verify figures against, so it instead evaluates whether the answer is financially plausible and internally consistent. For example, if an answer states a company's revenue grew 10% but then reports a lower absolute figure than the prior year, the judge would penalize that inconsistency even without knowing the true figures. The maximum score per answer is 3, giving a combined maximum of 9 across all three models per question and 180 across the full 20-question dataset.

Completeness measures whether the answer fully addresses all aspects of the question. A score of 3 indicates the answer covers everything asked, while lower scores reflect partial or superficial responses. The maximum follows the same structure as factual accuracy.

Conciseness measures whether the answer is free of unnecessary filler or irrelevant content, scored on a binary 0 or 1 scale. The maximum combined score is 3 per question and 60 across the dataset. After analyzing the results, this metric proved largely uninformative as a differentiator, since all three models are instruction-tuned to produce clean, focused responses by default. Conciseness is also orthogonal to factual quality, as a model can produce a concise but factually incorrect answer, which is consistent with what the LLM-only results show below. The metric is retained for completeness but should be interpreted with this limitation in mind.

4.4.3 Results Analysis

The aggregated scores across 20 questions are reported below for both evaluation modes:



Under the reference-anchored evaluation, RAG achieved a total score of 124/140, with strong performance across factual accuracy (51/60) and completeness (56/60). LLM-only scored **considerably lower** at 59/140, with **factual accuracy dropping sharply** to 10/60, reflecting the model's limited ability to produce precise financial figures without access to source documents. Web search scored 66/140, performing slightly better than LLM-only on factual accuracy (17/60) but similarly weak on completeness (29/60). Both baselines scored near-perfect on conciseness, reinforcing the point that lower overall scores are driven entirely by gaps in factual grounding and completeness rather than verbosity.

Under the reference-anchored evaluation, RAG achieved a total score of 124/140, with **strong performance** across factual accuracy (51/60) and completeness (56/60). LLM-only scored considerably lower at 59/140, with factual accuracy dropping sharply to 10/60, reflecting the model's limited ability to produce precise financial figures without access to source documents. Web search scored 66/140, performing slightly better than LLM-only on factual accuracy (17/60) but similarly weak on completeness (29/60). RAG scored the lowest on conciseness (17/20) across this mode, which is likely attributable to its inclusion of source citations and inline references. With a retrieved chunk present as the ground truth anchor, the judge may interpret citations that extend beyond the reference as extraneous and penalized conciseness even though they are a deliberate feature of the system. Both LLM-only and web search scored near-perfect on

conciseness, reinforcing that lower overall scores for the baselines are driven by gaps in factual grounding and completeness rather than verbosity.

One notable finding is that LLM-only outscores web search under the reference-free mode despite web search having access to live financial data. This is likely a reflection of answer construction rather than information quality. LLM-only responses tend to be cautiously worded and internally coherent, which scores well under a plausibility-based rubric. Web search responses cite specific figures from updated sources, and when those figures differ from what the judge considers plausible, or when multiple sources introduce slight inconsistencies in reporting periods or terminology, the judge may penalize them even if the figures are factually correct. This is consistent with the limitation noted earlier: without a reference anchor, the judge cannot distinguish between an unfamiliar correct figure and a hallucinated one, which means the reference-free mode may still undervalue web search in practice.

Across both modes, RAG consistently outperforms both baselines, lending support to the practical value of the retrieval pipeline as a complete system. The narrowing of the gap in the reference-free mode suggests that some portion of RAG's advantage in the reference-anchored evaluation reflects retrieval-source bias, though RAG still leads by a meaningful margin even when that bias is removed.